YAPL-S Lexer & Parser Implementation

YAPL Extended with Native String Operations
CS 327: Compilers

Group 6
21110202 21110060 22110099 21110210

Spring 2026

Contents

1 Introduction

YAPL-S is a variant of the YAPL language (a subset of ANSI 2011 C Standard) as taught in **CS327: Compilers** course at IITGN. It retains the core syntax and semantics of YAPL (language constructs, supported data types and supported operators), but additionally introduces native support for basic string manipulation as YAPL does not support `string` preprocessors.

This specification details the extensions made to the base YAPL language to support Pythonic **F-Strings** (Format Strings) and **String Concatenation** as native language features, eliminating the need for the `<string.h>` library for common string operations.

Note: The YAPL-S and the original YAPL language specifications are provided in Appendix A, B and C.

2 Construction of LALR(1) Automaton

Given the sheer scale of a complete C grammar (which typically generates hundreds of states), constructing the entire LALR(1) automaton by hand is computationally infeasible. Instead, we leveraged `yacc` with the verbose flag (`yacc -v -d yapl_s.y`) to automatically compute the LALR(1) item sets, generate the state machine, and identify parsing conflicts. The resulting automaton and its diagnostics are captured in the generated `yapl_s.output` file.

2.1 Construction of LALR(1) Item Sets

An item set represents a specific state in the parser. It contains the grammar productions currently being tracked, with a dot (· or .) indicating the parser's current position within that production (i.e., what has been seen so far versus what is expected next).

For example, observing State 153 (related to newly added F-Strings) from our generated `yapl_s.output` file:

```
State 153

    7 f_string_expression: FSTRING_START f_string_body . FSTRING_END
    9 f_string_body: f_string_body . STRING_LITERAL_PART
   10              | f_string_body . interpolation_block
```

In this item set, the parser has successfully matched the beginning of an F-String (`FSTRING_START`) and some arbitrary amount of `f_string_body`. The dot (.) indicates that the parser is currently inside the F-String and is waiting to evaluate the next token to determine if the string is ending, continuing with literal text, or opening an interpolation block.

2.2 Building the LALR(1) Automaton

The LALR(1) automaton is a directed graph where the item sets act as nodes (states) and the lookahead tokens act as the directed edges (transitions). The transition function is represented by the shift, reduce, and goto actions detailed in the `yapl_s.output` file.

Continuing with our F-String example in State 153, the automaton defines the following transitions based on the next input token:

```
    FSTRING_END          shift, and go to state 262
    INTERPOLATION_START  shift, and go to state 263
    STRING_LITERAL_PART  shift, and go to state 264

    interpolation_block  go to state 265
```

This indicates three distinct directed terminal edges (shifts) and one non-terminal edge (goto) originating from this state:

1. If the lookahead is `FSTRING_END`, the parser shifts and transitions to State 262, preparing to finalize the `f_string_expression`.

2. If the lookahead is `INTERPOLATION_START` (our { equivalent in F-Strings), it shifts to State 263 to begin parsing a nested C expression.

3. If the lookahead is `STRING_LITERAL_PART`, it shifts to State 264 to consume more standard string characters.

4. Once an `interpolation_block` is fully reduced by a later state, the parser will use the goto transition to move to State 265.

Note: The complete graphical representation of this automaton is not available since the number of states is almost 500 and visualizing it is infeasible.

2.3 Conflict Identification

Analyzing the LALR(1) automaton reveals exactly 3 Shift/Reduce conflicts and 0 Reduce/Reduce conflicts. Below is the explicit breakdown of each conflict, the states where they occur, and the exact productions involved. Noticeably, these conflicts existed in the original YAPL language and still exists in our extension.

```
State 27 conflicts: 1 shift/reduce
State 344 conflicts: 1 shift/reduce
State 466 conflicts: 1 shift/reduce
```

1. ATOMIC Specifier/Qualifier Ambiguity

```
State 27

  165 atomic_type_specifier: ATOMIC . '(' type_name ')'
  169 type_qualifier: ATOMIC .

    '('  shift, and go to state 49

    '('       [reduce using rule 169 (type_qualifier)]
    $default  reduce using rule 169 (type_qualifier)
```

   (a) State: 27

   (b) Type of Conflict: Shift/Reduce

   (c) Productions Involved:

   - Shift: `atomic_type_specifier : ATOMIC . '(' type_name ')' (Rule 165)`
   - Reduce: `type_qualifier : ATOMIC . (Rule 169)`

   (d) Explanation: When the parser encounters the `ATOMIC` keyword and peeks at the next token `(`, it faces an ambiguity. It does not know whether to shift the `(` to build an `atomic_type_specifier` (e.g., `_Atomic(int)`), or to immediately reduce `ATOMIC` into a `type_qualifier` (e.g., treating it like a standard qualifier in `_Atomic int x;`).

2. Mid-Rule Action (Epsilon) Ambiguity

```
State 344

  175 declarator: pointer . $@1 direct_declarator
  208 abstract_declarator: pointer . direct_abstract_declarator
  209                    | pointer .
```

```
    '('  shift, and go to state 188
    '['  shift, and go to state 189

    IDENTIFIER  reduce using rule 174 ($@1)
    '('         [reduce using rule 174 ($@1)]
    $default    reduce using rule 209 (abstract_declarator)

    $@1                          go to state 136
    direct_abstract_declarator   go to state 315
```

(a) State: 344

(b) Type of Conflict: Shift/Reduce

(c) Productions Involved:

- Shift: `abstract_declarator : pointer . direct_abstract_declarator (Rule 208)`
- Reduce: `declarator : pointer . $@1 direct_declarator (Rule 175)`

(d) Explanation: This conflict arises from an embedded semantic action (`{pointer_decls++;}`) inside the declarator rule. Yacc translates mid-rule actions into invisible, empty (epsilon) rules (represented here as `$@1`). When the parser sits at the pointer and sees a lookahead like `(`, it must decide whether to reduce the empty rule to execute the counter (committing to a declarator), or to shift the `(` to continue building an `abstract_declarator`.

3. Dangling Else Ambiguity

```
State 466

  263 selection_statement: IF '(' expression ')' statement . ELSE
   $@2 statement
  264                    | IF '(' expression ')' statement .

    ELSE  shift, and go to state 481

    ELSE      [reduce using rule 264 (selection_statement)]
    $default  reduce using rule 264 (selection_statement)
```

(a) State: 466

(b) Type of Conflict: Shift/Reduce

(c) Productions Involved:

- Shift: `selection_statement : IF '(' expression ')' statement . ELSE $@2 statement (Rule 263)`
- Reduce: `selection_statement : IF '(' expression ')' statement . (Rule 264)`

(d) Explanation: This is the standard dangling-else ambiguity inherent in C-like grammars. When the parser completes an `IF` block and sees an `ELSE` token on the lookahead, it cannot structurally determine whether to shift the `ELSE` (attaching it to the closest/inner `IF`) or to reduce the current rule (closing the inner statement and attaching the `ELSE` to an outer `IF`).

3 Conflict Resolution

3.1 Grammar Modifications & Architecture Overhaul

While analyzing the initial Shift/Reduce conflicts—specifically the ambiguity surrounding the `ATOMIC` keyword in State 27—we made a critical architectural realization. The `ATOMIC` keyword, along with several other K&R and C99 legacy structures, were not part of the intended YAPL-S language specification. The presence of rules for these unsupported constructs was artificially bloating the parser and generating "ghost conflicts."

Consequently, we executed a major architectural overhaul.

Major Grammar Changes

Instead of forcing a resolution on the `ATOMIC` conflict, we traced every unsupported token (e.g., `_Atomic`, `restrict`, legacy K&R function declarations) through the grammar and systematically excised their corresponding productions from both `yapl_s_new.l` and `yapl_s_new.y`.

While the sheer volume of these targeted removals is too extensive to list line-by-line, the result was a dramatic optimization of the LALR(1) automaton. The total number of states was reduced from approximately 500 down to roughly 300, permanently eliminating the State 27 conflict by completely removing the ambiguous branches.

Resolving the Mid-Rule Action

For legitimate conflicts involving supported constructs, we applied structural grammar modifications. To fix the epsilon (ϵ) conflict in the `declarator` rule (State 344), we delayed the execution of the semantic action by moving it to the end of the production.

```
/* BEFORE: declarator: pointer {pointer_decls++;} direct_declarator */

/* AFTER: Semantic action deferred to the end */
declarator
    : pointer direct_declarator {pointer_decls++;}
    | direct_declarator
    ;
```

Resolving the Dangling-Else

To resolve the inherent Dangling-Else ambiguity without rewriting the core structure of C-style selection statements, we defined explicit token precedences to force the parser's hand.

```
/* Precedence rules */
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

/* Applied to the grammar */
selection_statement
    : IF '(' expression ')' statement ELSE statement
        {
            TRACE_REDUCE("selection_statement -> IF '(' expression ')'
   statement ELSE statement");
        }
    | if_without_else
        {
            TRACE_REDUCE("selection_statement -> if_without_else");
        }
    | SWITCH '(' expression ')' statement
```

```
        {
            TRACE_REDUCE("selection_statement -> SWITCH '(' expression
    ')' statement");
        }
    ;

if_without_else
    : IF '(' expression ')' statement %prec LOWER_THAN_ELSE
        {
            TRACE_REDUCE("if_without_else -> IF '(' expression ')'
    statement");
        }
    ;
```

Re-compiling the overhauled grammar with `yacc -v -d yapl_s_new.y` yields exactly **0 conflicts** and a highly optimized parsing table.

3.2 Adversarial Testing of Conflicts

When LALR(1) conflicts exist, Yacc employs default heuristics to forcefully resolve the ambiguity and generate executable code (typically defaulting to a `shift` action during a Shift/Reduce conflict). To verify our grammar modifications, we compiled and executed both the original ambiguous grammar and our overhauled deterministic grammar against three specific adversarial edge cases.

The ATOMIC Ambiguity

```
int main() {
    /* Should evaluate the State 27 ATOMIC ambiguity in old grammar */
    _Atomic(int) counter;
    return 0;
}
```

- **Original Compiler:** The original `yapl_s.y` triggers a Shift/Reduce conflict for the `ATOMIC` keyword. However, upon executing the adversarial code, the parser immediately throws a `syntax error, unexpected IDENTIFIER`. This reveals a fundamental desynchronization between the provided Lexer and Parser: the lexer lacked a definition for `_Atomic`, meaning the `ATOMIC` token was mathematically unreachable. The conflict was a "Ghost Conflict" existing in dead code.
- **New Compiler:** Realizing that the conflict was unreachable and the feature was unsupported by the YAPL-S specification, we completely amputated the `ATOMIC` productions from the Yacc grammar. The new compiler cleanly catches `_Atomic` as an unsupported identifier at the lexer level and rejects it.

Mid-Rule Action Ambiguity

```
int main() {
    /* Triggers the State 344 epsilon conflict */
    int (*function_ptr)(int, int);
    return 0;
}
```

- **Original Compiler:** The parser successfully generated an AST (`***parsing successful***`). However, it only achieved this because Yacc's internal conflict-resolution heuristic defaults to a `shift` action when encountering the ambiguous ϵ-transition caused by the mid-rule C-code. The success was a byproduct of toolchain guessing at generation time, not grammar accuracy.
- **New Compiler:** The parser successfully generated the exact same AST, but with **zero Shift/Reduce conflicts during parser generation**. By relocating the semantic action to the end of the production, the parser shifted the `(` token deterministically, proving the state machine is mathematically sound without relying on fallback heuristics.

Dangling-Else Ambiguity

```c
int main() {
    int x = 1, y = 1, z = 0;

    /* Triggers the State 466 Dangling-Else conflict */
    if (x > 0)
        if (y > 0)
            z = 1;
    else
        z = 2;

    return 0;
}
```

- **Original Compiler:** The execution output yields `***parsing successful***`. The parser correctly bounds the `else` block to the innermost `if`. However, this runtime success masks an underlying generation-time ambiguity: the original grammar produced a Shift/Reduce conflict during the `yacc` build phase. The correct AST was only constructed because Yacc's hardcoded fallback heuristic always defaults to a `shift` operation, which coincidentally aligns with standard C scoping rules.
- **New Compiler:** The execution output identically yields `***parsing successful***`. The crucial architectural difference lies in the parser generation phase: the new grammar builds with **zero Shift/Reduce conflicts**. By explicitly applying the `%prec LOWER_THAN_ELSE` directive, the shift action is mathematically codified into the LALR(1) state machine. The parser structurally guarantees the correct C scoping rules without relying on any silent toolchain heuristics or default guesses.

4 Reverse Derivation Tree

Bottom-up parsers, including LALR(1) parsers generated by Yacc, operate by shifting input tokens onto a stack and reducing them to non-terminals when a right-hand side production is matched. The chronological sequence of these `reduce` operations corresponds to a **Rightmost Derivation in reverse**. To output the standard top-down derivation tree (from root to leaves), the parser must record every reduction and invert the sequence upon completion.

To transform the flat, 300-rule bottom-up output into a professional, hierarchically ordered Abstract Syntax Tree (AST), we implemented a multi-stage data pipeline across the Lexer (`yapl_s.l`) and Parser (`yapl_s_new.y`).

4.0.1 Lexer Changes

By default, the global `yytext` buffer is continuously overwritten as the Lexer scans new tokens. Because parser reductions occur asynchronously (often several tokens *after* a lexeme is scanned), directly printing `yytext` during a reduction yields incorrect data. To preserve the original source code mapping, we renamed the Lex scanning routine to `real_yylex()` and injected a custom wrapper function inside Yacc:

```
/* Interceptor Wrapper */
int yylex(void) {
    int tok = real_yylex();
    if (tok > 0 && yytext != NULL) {
        lexeme_list[lexeme_count++] = sanitize_for_dot(yytext);
    }
    return tok;
}
```

This safely queues every exact lexeme scanned into an ordered array before the token is consumed by the parser.

4.0.2 Parser Changes

Instead of relying on `printf` logs, the `TRACE_REDUCE` macro was upgraded to store the raw string of every triggered rule into a dynamic array (`derivation_tree`).

Upon a successful parse, we iterate through this array in reverse order. We tokenize the left-hand and right-hand sides of the stored rules and dynamically allocate `TreeNode` structs. To handle Yacc's hidden mid-rule actions (which create dummy ϵ-transitions that normally desynchronize stack pops), we implemented a stack matching algorithm. Rather than blindly popping the stack, the algorithm searches the unexpanded non-terminal stack for exact string matches, cleanly ignoring phantom Yacc nodes and preserving the structural integrity of the tree.

4.0.3 Graphviz Integration

To ensure the resulting tree was human-readable, the AST is exported directly into the `.dot` language for Graphviz rendering.

- **Strict Horizontal Ordering:** Graphviz natively tangles deep trees to balance aesthetic weight. We forced a strict left-to-right code mapping by applying `ordering="out"` and using invisible structural edges (`-> [style=invis]`) inside a { `rank=same;` } block to lock all terminal leaves to the bottom baseline.
- **Post-Processing:** Before export, a recursive function traverses the leaves from left to right. It actively filters out unexpanded ϵ-rules (Phantom Leaves) by detecting lowercase non-terminal identifiers. It then attaches the saved source-code strings from the

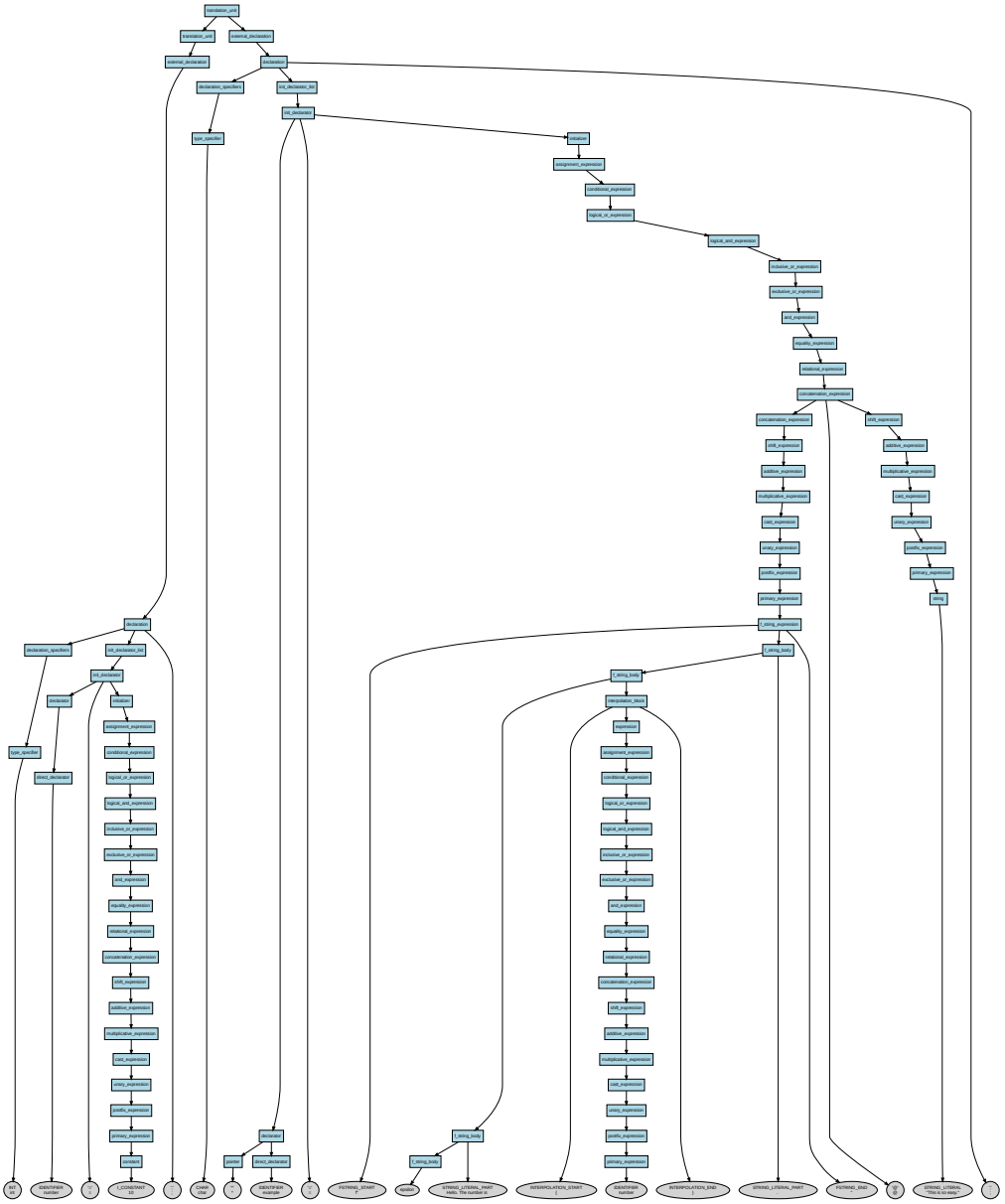


Figure 1: Reverse Derivation Tree of the above program

`lexeme_list` exactly onto the validated terminal nodes (e.g., mapping `"Alice"` directly to the `STRING_LITERAL` leaf).

The result is a mathematically complete, strictly ordered, high-resolution SVG visual map of the program's rightmost derivation.

We show the reverse derivation tree for a small C program in Figure 1.

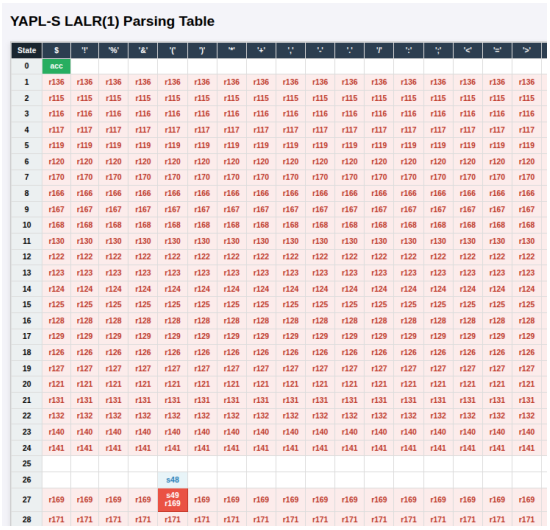
```
int number = 10;
char *example = f"Hello. The number is {number}. " @ "This is so easy.";
```

5 LALR(1) Parsing Table

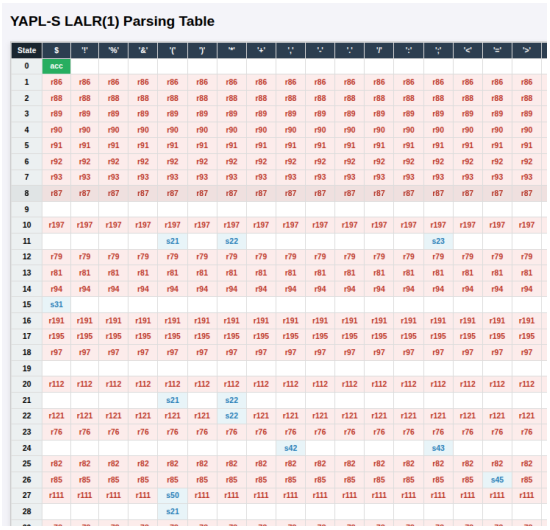
To generate LALR(1) Parsing Table, we again relied on the verbose output files generated in Section 2, carefully parsing the shift, reduce and goto actions for each state.

A partial comparison of the resulting LALR(1) tables is shown in Figure 2. Figure 2a visualizes the original Shift/Reduce conflict in State 27 of the old grammar, which was the primary catalyst for the re-design. Figure 2b shows a successful segment of the new, optimized parsing table, which is entirely conflict-free.

The complete parsing tables for both the original and new grammar are attached to this report as standalone HTML files, `parsing_table_old.html` and `parsing_table_new.html`. We recommend viewing these files in a web browser.



(a) Conflict in State 27 of Old Grammar (`yapl_s`)



(b) Conflict-Free Portion of New Grammar (`yapl_s_new`) Parsing Table

Figure 2: Comparison of original parsing conflict versus new deterministic table.

6 Error Diagnosis

By default, standard Yacc implementations halt immediately upon encountering an invalid token sequence and emit a generic, unhelpful string: `syntax error`. In addition to this, we implemented code to pinpoint the exact spatial location and grammatical nature of a syntax violation.

The diagnostic system required modifications across both the Lexer and the Parser.

1. **Lexer Changes:** We enabled `%option yylineno` within Lex to automatically track the active line number. However, column tracking requires manual intervention. To accurately compute column numbers across multi-line comments and whitespace blocks, we defined a custom `YY_USER_ACTION` macro. This macro intercepts the lexer prior to executing any token's semantic action, iterates through the matched `yytext` buffer, and dynamically advances or resets a global `yycolumn` counter.

2. **Verbose State Exposure:** We injected the `%define parse.error verbose` directive into the Yacc specification. Instead of simply failing, this directive instructs the LALR(1) automaton to cross-reference its current state against the lookahead table upon a failure, allowing it to explicitly list the expected tokens versus the offending token.

To demonstrate the diagnostic enhancement, we fed both the original and modified compilers a C program with a fault.

Example Program:

```
int main() {
    int x = 10;
    // Missing the closing '}' for the interpolation block
    char *broken = f"The value is { x + 5 ";

    return 0;
}
```

Original Compiler Output:

```
***parsing terminated*** [syntax error]
```

New YAPL-S Compiler Output

```
=====================================================
                  COMPILER ERROR
=====================================================
 Location : Line 4, Column 44
 Token    : ';'
 Message  : syntax error, unexpected ';', expecting INTERPOLATION_END or
    ','
=====================================================
***parsing terminated*** [syntax error]
```

7 Shared Artifacts

Along with this report, we have provided the complete source code repository for the YAPL-S: Github Repo

Using the provided `Makefile`, one can generate both the original and updated lexers and parsers to test the behavioral differences. Detailed build instructions and execution commands are available in the dedicated `README.md` file.

The repository is organized into the following key components:

- `yapl_s.l` and `yapl_s.y`: The original, unoptimized Lex and Yacc files containing the unsupported legacy constructs and documented Shift/Reduce conflicts.
- `yapl_s_new.l` and `yapl_s_new.y`: The overhauled, strictly deterministic grammar files.
- `parser_analysis/`: This includes the `generate_table.py` script used to construct the interactive HTML parsing tables, and the `adversarial_tests/` subdirectory containing the specific C snippets and execution logs used to verify the conflict resolutions.
- `tests/`: A comprehensive evaluation suite categorized into `positive/`, `negative/`, and `misc/` directories. Each subdirectory contains test C files alongside side-by-side execution logs (`output_old_*.txt` vs. `output_new_*.txt`) and generated Reverse Derivation Trees in SVG format for the successfully-compiled programs using the new compiler.

A Appendix: YAPL-S Language Specification

A.1 Lexical Conventions

The lexical analyzer distinguishes tokens based on the standard C rules, with the following additions for handling string interpolation.

A.1.1 String Concatenation Operator

The following operator is added to the language:

Operator	Symbol	Description
Concatenation	`@`	Binary operator for joining two string literals.

Table 1: New Operators in YAPL-S

A.1.2 Interpolated String Literals

An interpolated string literal (Pythonic F-String) is distinguished from a standard string literal by the prefix `f`.

Syntax:

```
f" text { expression } text "
```

The lexical analyzer would process F-Strings using the `lex` tool to emit the following distinct token types:

- `FSTRING_START`: The sequence `f"`.
- `STRING_LITERAL_PART`: A sequence of characters inside the F-String that are *not* part of an interpolation.
- `INTERPOLATION_START`: The { character inside an F-String.
- `INTERPOLATION_END`: The } character inside an F-String.
- `FSTRING_END`: The closing " character.

Note: Standard escape sequences (e.g., \n, \") are valid within `STRING_LITERAL_PART`.

A.2 Syntax

The grammar of YAPL-S extends the YAPL grammar.

A.2.1 Operator Precedence and Associativity

The `@` operator is inserted strictly between the **Shift** and **Relational** tiers. This ensures `char *` arithmetic (addition/subtraction) takes priority over concatenation.

Prec.	Operator Class	Operators	Associativity
1	Primary/Postfix	`() [] -> .`	L-to-R
2	Unary	`* & - !`	R-to-L
3	Multiplicative	`* / %`	L-to-R
4	Additive	`+ -`	L-to-R
5	Shift	`<< >>`	L-to-R
6	**Concatenation**	`@`	**L-to-R**
7	Relational	`< > <= >= <=>`	L-to-R
8	Equality	`== !=`	L-to-R

Table 2: Revised Operator Precedence Table

A.2.2 Grammar Specification

The grammar hierarchy reflects the precedence table.

```
// 1. Relational expressions derive from concatenation
relational_expression
  : concatenation_expression
  | relational_expression '<' concatenation_expression
  | relational_expression '>' concatenation_expression
  | relational_expression LE_OP concatenation_expression
  | relational_expression GE_OP concatenation_expression
  | relational_expression TH_OP concatenation_expression
  ;

// 2. Concatenation derives from Shift
// This ensures @ has lower precedence than <<, +, *, etc.
concatenation_expression
    : shift_expression
    | concatenation_expression '@' shift_expression
    ;

// 3. Shift derives from Additive (Standard C)
shift_expression
  : additive_expression
  | shift_expression LEFT_OP additive_expression
  | shift_expression RIGHT_OP additive_expression
  ;

// 4. Primary Expression updated for F-Strings
primary_expression
  : IDENTIFIER
  | constant
  | string
  | '(' expression ')'
  | generic_selection
```

```
    | f_string_expression   /* Extension */
  ;

// 5. F-String Structure
f_string_expression
    : FSTRING_START f_string_body FSTRING_END
    ;

f_string_body
    : /* empty */
    | f_string_body STRING_LITERAL_PART
    | f_string_body interpolation_block
    ;

interpolation_block
    : INTERPOLATION_START expression INTERPOLATION_END
    ;
```

A.3 Semantics

A.3.1 F-String Evaluation

An F-String expression evaluates to a value of type `char *`.

1. **Evaluation Order:** The expressions inside the interpolation blocks $\{\ldots\}$ are evaluated from left to right at runtime.
2. **Type Conversion:** The result of each interpolated expression is implicitly converted to its string representation.
   - `int`: Converted to decimal string (e.g., `10` $\rightarrow$ `"10"`).
   - `float/double`: Converted to standard floating-point representation.
   - `char *`: Copied as-is.
   - `char`: Converted to a single-character string.
3. **Construction:** The literal text parts and the converted expression strings are concatenated in order.
4. **Memory:** The result is stored in a newly allocated memory block (heap). The runtime environment manages this allocation.

A.3.2 The Concatenation Operator (`@`)

The `@` operator performs string concatenation.

- **Operands:** Both operands must evaluate to type `char *`.
- **Result:** A new `char *` containing the contents of the left operand followed immediately by the right operand.
- **Error Handling**: The compiler shall issue a Type Error if an operand is numeric (e.g., `str @ 5` is illegal).
- **Behavior**: Resultant string is automatically null-terminated.
  1. Compute length of left string (L_1) and right string (L_2).
  2. Allocate $L_1 + L_2 + 1$ bytes on the heap.
  3. Copy Left String $\rightarrow$ Copy Right String $\rightarrow$ Null Terminate.

- **Rationale for Precedence**: Given `str1 @ str2 + 1`:
  - Additive (+) runs first: `str2 + 1` advances the pointer by 1 char.
  - Concatenation (@) runs second: Joins `str1` with the substring.
  - This prevents the inefficiency of allocating a new string only to immediately skip its first character.

A.4 Examples and Use Cases

A.4.1 Initialization and Assignment

Native strings can be created dynamically without reliance on `sprintf` or `strcat`.

```
int main() {
    int id = 42;
    double score = 98.5;
    char *user = "Alice";

    // --- Standard C11 Approach (Tedious) ---
    // Requires <stdio.h>, <stdlib.h>, <string.h>
    // char buffer[100];
    // sprintf(buffer, "User: %s (ID: %d)", user, id);

    // --- YAPL-S Approach (Native) ---
    // F-Strings handle implicit memory allocation and formatting
    char *status = f"User: {user} (ID: {id})";

    // String Concatenation using the @ operator
    char *full_msg = status @ " - Status: Active";

    // full_msg is now: "User: Alice (ID: 42) - Status: Active"

    return 0;
}
```

A.4.2 Usage in Function Calls

F-Strings evaluate to `char *`, allowing them to be passed directly to any function expecting a C string.

```
void log_event(char *msg) {
    // Implementation for logging...
}

int main() {
    int x = 10, y = 20;

    // Direct usage in printf (replaces %s format specifiers)
    printf(f"Coordinates: x={x}, y={y}\n");

    // Usage in custom functions with embedded arithmetic
    // The expression {x + y} is evaluated before string construction
    log_event(f"System Check: {x + y} units detected.");

    return 0;
}
```

A.4.3 Interaction with Pointer Arithmetic

Because `@` has lower precedence than additive operators (`+`, `-`), standard C pointer arithmetic behaves predictably when mixed with concatenation.

```
int main() {
    char *h = "Hello";
    char *w = "World";

    // Scenario A: Pointer Arithmetic on the Operand (Standard
   Precedence)
    // 1. ("World" + 1) advances pointer to "orld"
    // 2. "Hello" @ "orld" concatenates
    char *subset = h @ w + 1;
    // Result: "Helloorld"

    // Scenario B: Pointer Arithmetic on the Result (Grouped)
    // 1. ("Hello" @ "World") creates "HelloWorld"
    // 2. (+ 1) advances pointer on the new string
    char *offset = (h @ w) + 1;
    // Result: "elloWorld"

    return 0;
}
```

A.4.4 Nested Expressions and Escaping

Based on the implementation, F-strings can be nested arbitrarily deep, enabling complex formatting logic.

```
int main() {
    int val = 100;

    // Nested F-String:
    // The inner expression { f"[{val}]" } evaluates to "[100]" first.
    char *debug = f"Debug Info: { f"Value -> [{val}]" }";

    return 0;
}
```

A.4.5 Compiler Error Handling

The compiler enforces strict type safety for the `@` operator to prevent accidental pointer miscalculations.

```
int main() {
    char *base = "Error Code: ";
    int code = 404;

    // INVALID: Cannot concatenate string (char *) with integer (int)
    // char *err = base @ code;  <-- Compilation Error

    // CORRECT: Use F-String to convert the integer first
    char *msg = base @ f"{code}";
    // Result: "Error Code: 404"

    return 0;
}
```

B Appendix: YAPL-S - Lexer and Parser Architecture Guide

In this document, we will go through the architectural additions and internal mechanisms used to extend the base YAPL compiler into YAPL-S. It assumes familiarity with the YAPL-S language specification and focuses on how the new features were implemented in Lex (`yapl_s.l`) and Yacc (`yapl_s.y`), with a special emphasis on the **state-based tokenizer**.

B.1 Architectural Delta: YAPL vs. YAPL-S

The base YAPL language is a standard subset of C11 that lacks native string manipulation capabilities. YAPL-S introduces two major semantic features requiring deep modifications to both the lexer and the parser:

- **Binary String Concatenation (`@`)**:
  - **Lexer Addition**: A simple pass-through token for `@`.
  - **Parser Addition**: A new `concatenation_expression` tier was injected strictly between `shift_expression` and `relational_expression`. This specifically preserves standard C pointer arithmetic precedence (e.g., `ptr + 1` happens before concatenation).
- **Interpolated F-Strings (`f"..."`)**:
  - **Lexer Addition**: A multi-state, stack-based tokenization logic to handle *context-sensitive scanning.*
  - **Parser Addition**: A left-recursive grammar rule that combines text literals and C-expressions.

B.2 YAPL-S Lexer: Context-Sensitive Tokenization

Standard Lex is a **Finite State Automaton (FSA)**. FSAs have no memory and cannot balance nested structures.

If we parse an F-string like `f"Value:{x + 1}"`, the lexer encounters a severe conflict:

- In String (dubbed String/`FSTRING` Mode hereafter), a space is a literal character that must be kept. The word `int` is just the letters i-n-t.
- Inside the {} (dubbed Code/`INITIAL` Mode hereafter), a space is whitespace to be ignored. The word `int` is a language keyword.

Because Lex operates ahead of the Yacc parser, Yacc cannot tell Lex how to interpret the characters. The Lexer itself must maintain its own context. This led us to *Context-Sensitive Lexing.*

B.2.1 Python PEP 701: Syntactic formalization of f-strings

The problem of nested interpolation is not unique to YAPL-S. Python faced this exact limitation. Before Python 3.12, the CPython lexer used a "hack" where it read the entire `f"..."` as a single, massive string token, deferring the extraction of the inner `{...}` expressions to the parser. This meant you could not easily nest f-strings or use the same quote types inside them.

In PEP 701, Python completely overhauled its lexer to use a State Stack.

> "This PEP proposes formalizing the f-string syntax... The parser will be modified to understand f-strings natively. This is achieved by adding a new state to the tokenizer... When the tokenizer finds an f-string, it enters a specific mode... When it finds a brace { inside an f-string, it drops back to the regular mode, but keeps track of the fact that it was inside an f-string using a stack." - Python PEP 701

YAPL-S adopts this exact modern architecture, effectively bypassing the limitations of older string interpolation designs.

B.2.2 YAPL-S Stack-Based Lexer Implementation

To replicate the PEP 701 behavior in standard Lex, the YAPL-S lexer creates a simulated *Pushdown Automaton (PDA)* using a C-array stack.
Core Components in `yapl_s.l`:

- **State Stack**: An array `int state_stack[100]` with `push_state()` and `pop_state()` functions. This remembers the "modes" the lexer was in.
- **Added New Start Condition (`%x FSTRING`)**: An exclusive lexical state. When active, normal C code rules (like keywords or ignoring whitespace) are completely disabled. Used to scan string literals as intended.
- **Brace Counter (`brace_depth`)**: An integer that counts nested { and } while in normal C Code Mode.

How the Modes Switch:

- **Entering**: When Lex sees `f"`, it saves its current state to the stack and executes `BEGIN(FSTRING)`. Now, it tokenizes string literals as is.
- **Interpolating**: When Lex sees { inside an F-string, it pushes `FSTRING` to the stack, resets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. Now, it perfectly tokenizes C code.
- **Exiting**: When Lex sees } in `INITIAL` mode, it checks `brace_depth`. If `brace_depth == 0` and the top of the stack is `FSTRING`, it knows this brace closes an interpolation, not a C Code block. It pops the stack and executes `BEGIN(FSTRING)`.

B.2.3 The `<FSTRING>` State and Token Rules

To safely parse string contents without accidentally triggering standard C keywords (like `if` or `while`), the lexer defines an exclusive start condition via `%x FSTRING`. When the lexer enters this state, it ignores all normal C rules and operates inside a strict "string sandbox."

Within this sandbox, the lexer evaluates a highly specific set of rules to break the string down into its component tokens:

- **Raw Text Consumption**: The most important regular expression of the String Mode is `([^"{}\\\n]|{ES})+`. It rapidly gobbles up continuous blocks of standard text and valid C-escape sequences (like `\n` or `\t`), stopping only when it hits a boundary character: a quote, a brace, a newline, or an invalid backslash.
- **Escaping Literal Braces**: YAPL-S mirrors modern standards (like Python) by using brace doubling. If the lexer encounters {{ or }} while inside the `<FSTRING>` state, it explicitly matches them via the `"{{"|"}}"` rule and emits them as safe `STRING_LITERAL_PART` tokens. This prevents the lexer from falling into `INITIAL` mode, allowing programmers to safely write strings like:

```
char *set = f"Math Set: {{1, 2, {dynamic_val}}}";
```

- **Interpolation Entry**: If a single { is encountered, the sandbox pauses. The lexer pushes the `FSTRING` state to the stack, sets `brace_depth = 0`, and executes `BEGIN(INITIAL)`. This hands control back to the standard C-lexer rules, allowing the inner expression to be tokenized normally. It simultaneously emits an `INTERPOLATION_START` token for the parser.

- **String Termination**: When the lexer encounters an unescaped ¨, it knows the current string segment is complete. It pops the previous state from the stack (using `pop_state()`), executes a `BEGIN` to return to that state, and emits `FSTRING_END`.
- **Strict Error Handling**: To prevent runaway parsing, the sandbox contains strict error-catching rules:
  - **Unescaped Newlines**: F-strings, like standard C-strings, are not allowed to silently span multiple lines. Catching a raw `\n` triggers a `[lexer error]: unescaped newline in f-string`.
  - **Invalid Escapes**: A stray backslash (\) that does not form a valid escape sequence instantly halts the lexer with an `invalid escape sequence` error.

B.3 YAPL-S Parser

To support the new string features without breaking standard ANSI C11 behavior, the YAPL-S Yacc grammar required two major architectural injections.

B.3.1 The Concatenation Operator Precedence Tier

The base YAPL language groups operators into strictly ordered tiers (e.g., `multiplicative_expression`, `additive_expression`, `shift_expression`).

To safely introduce the `@` operator, a new `concatenation_expression` tier was inserted directly between `shift_expression` and `relational_expression`.

```
concatenation_expression
    : shift_expression
    | concatenation_expression '@' shift_expression
    ;
```

By making concatenation derive from shift (and ultimately additive) expressions, the parser intrinsically guarantees that C pointer arithmetic (like `str + 1`) resolves before string concatenation.

B.3.2 Left-Recursive String Assembly

When the lexer breaks an F-string into chunks, the parser must stitch them back together dynamically. This is achieved using a left-recursive grammar rule for `f_string_body`.

```
f_string_body
    : /* empty */
    | f_string_body STRING_LITERAL_PART
    | f_string_body interpolation_block
    ;
```

Left recursion is highly efficient for LALR parsers like Yacc. It ensures the parser never has more than two or three string chunks on its internal stack at any given time. As each new `STRING_LITERAL_PART` or `interpolation_block` arrives, the parser immediately reduces it into the growing `f_string_body` node.

Note: Refer to the YAPL-S language specifications as provided in Appendix A to view all the changes in the grammar.

Input	Lexer Mode (Before → After)	Lexer Stack (After Push/Pop)	Token Emitted	Parser Action
f"	INITIAL → FSTRING	[INITIAL]	FSTRING_START	**Shift** to Parse Stack.
A_	FSTRING	[INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** against empty f_string_body to start building.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. We are now parsing C code.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Lexer discards whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	**Shift**. Starting the first inner F-string.
{	FSTRING → INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. Preparing to parse the inner variable.
b	INITIAL	[INITIAL, FSTRING, INITIAL, FSTRING]	IDENTIFIER	**Shift** then **Reduce** ID → primary_expr → expr.
}	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the inner interpolation_block.
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	**Shift** then **Reduce** the entire inner f_string_expression.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Lexer discards whitespace again.
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the first outer interpolation_block and append it.
_	FSTRING	[INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** to append the space between the blocks.
{	FSTRING → INITIAL	[INITIAL, FSTRING]	INTERPOLATION_START	**Shift**. Opening the second outer interpolation block.
_	INITIAL	[INITIAL, FSTRING]	*(Ignored)*	Whitespace in code mode.
f"	INITIAL → FSTRING	[INITIAL, FSTRING, INITIAL]	FSTRING_START	**Shift**. Starting the second inner F-string.
C	FSTRING	[INITIAL, FSTRING, INITIAL]	STRING_LITERAL_PART	**Shift** then **Reduce** to an inner f_string_body.
"	FSTRING → INITIAL	[INITIAL, FSTRING]	FSTRING_END	**Shift** then **Reduce** to an f_string_expression.
}	INITIAL → FSTRING	[INITIAL]	INTERPOLATION_END	**Shift** then **Reduce** the second outer interpolation_block and append.
"	FSTRING → INITIAL	[] (Empty)	FSTRING_END	**Shift**. The outer F-string is complete.

Table 3: Step-by-Step Lexer and Parser Trace for Nested F-String Evaluation

B.4 Tracing an Example Nested F-String

Let's trace a highly complex, deeply nested edge case: `f"A␣{␣f"{b}"␣}␣{␣f"C"}"`, where ␣ denotes a space (for better readability).

This string contains an outer F-string, which contains two interpolation blocks. The first interpolation block contains another F-string, which itself interpolates a variable `b`. Furthermore, there is intentional spacing inside the code blocks which the lexer must know to ignore.

Refer to Table 3 for a step-by-step trace of how the lexer and parser collaborate to evaluate this sequence.

B.5 Shared Artifacts

Along with this document, we have attached the YAPL-S Lex (`yapl_s.l`) and Yacc (`yapl_s.y`) file. Using the `Makefile`, one can generate the lexers and parsers of YAPL-S and start tokenizing YAPL-S codes. Details of running the same are given in a dedicated `README.md` file.

We have also provided a test suite which contains both `positive` (cases where YAPL-S must successfully parse the code) and `negative` (cases where YAPL-S must fail parsing/lexing gracefully) examples. The outputs of the tokenizer along with some important grammar reductions are demonstrated as debug output in the corresponding `.txt` files.

The same can be found here: Github Repo

C Appendix: Base YAPL Language Specification

This appendix details the specification for the base YAPL language (a subset of C11) as available here: YAPL Language Specification (PDF).

C.1 Language Constructs

The language conforms to the ANSI C-2011 standard with the following specific inclusions:

- global variables, both normal as well as `extern`.
- function declaration, function definition, and function call (including recursion).
- `for` loop, `while` loop, `do-while`.
- `if`, `if-else`, `if-else-if` (nested/ladder) (if condition can be any expression including literals/constants).
- `switch/case` with case labeled with `:`; must support fall-through; must support the optional `default` clause.
- variable declaration, definition, initialization locally/globally (multiple variables comma-separated).
- strings (of the forms `"..."` (single), `"..." "..."` (multiple)), character, integer and floating point literals.
- single-line (`//`) and multi-line (`/*...*/`) comments (multi-line comments should be properly closed lexically).
- `break`, `continue`, and `return` statements.
- a statement must end in a semi-colon `;` like the usual standard of C.

C.2 Supported Data Types

- **Primary:** `int`, `long`, `short`, `float`, `double`, `void`, `char` and their pointers.
- **User-defined:** structures (`struct`) (pointer dereferences must support arbitrary level of indirection).
- **Derived:** functions, arrays, and pointers (all supported data types; no need to support function pointers).

C.3 Supported Operators

Operator precedence and associativity follow standard C rules.

- **Relational operators:** `<`, `>`, `==`, `<=`, `>=`, `!=`, `<=>`.
- **Unary operators:** `+`, `-`, `!`, `*`, `&`, `~`.
- **Binary operators:** `+`, `-`, `*`, `&`, `|`, `/`, `%`, `^`.
- **Logical operators:** `&&`, `||`.
- **Assignment operator:** `=`.
- **Suffix/postfix increment and decrement:** `++`, `--`.

- **Structure field access operators:** `->`, `.`.
- **Pointers:** `*`, `&` (pointer dereferences must support arbitrary/multiple level of indirection, i.e., `int *****p;`).
- **Function call:** `()` (any valid expression inside including blank, constants, and literals).
- **Array subscripting:** `[]` (any valid expression inside including constants and literals).
- `sizeof()`: operands: `<typename>` or unary expression.
- **Typecast:** `(<typename>)`.

C.4 Unsupported Constructs

The following C11 features are **explicitly excluded**:

- **Preprocessors:** none allowed (no `#include`s, `#define`s needed anywhere).
- **Operators:** `...` (ellipsis), `?:` (ternary operator) (function vararg list specification not required).
- **Operators:** `>>=`, `<<=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `^=`, `|=`, `<<`, `>>`.
- **Constructs/keywords:** `auto`, `const`, `goto`, `inline`, `register`, `restrict`, `signed`, `static`, `typedef`, `unsigned`, `enum`, `union`, `volatile`, `_Alignas`, `_Alignof`, `_Atomic`, `_Bool`, `_Complex`, `_Generic`, `_Imaginary`, `_Noreturn`, `_Static_assert`, `_Thread_local`, `__func__`.